Executive Summary: Automation Portfolio

Miguel Villarreal · Finance operations → automation · Python · SQL · LLMs · .NET · México

Repository: github.com/MVillarreal98/fde-portfolio

WHAT THIS IS

Four demos on billing and payment problems, with synthetic data. No project has users or a production deployment, and no figure below measures business impact: they are test counts and measurements; each README gives command, machine and versions.

PROJECTS: PROBLEM → ROOT CAUSE → SOLUTION → EVIDENCE

Project	Problem and root cause	Solution	Evidence in the repo
Reconciliation Engine (Python, pandas, DuckDB SQL)	A ledger and a settlement file disagree. Failure modes it guards against (several found in its own v1.0.0): no per-reference aggregation, IDs read as numbers, malformed amounts dropped silently.	Text-only ingestion, integer cents, one-to-one join with one status per reference, 7 rules, rejected-rows file, conservation checks, exit codes. Same rules in pandas and DuckDB SQL.	171 tests, 100% statement and branch coverage on pandas 2.3.3 and 3.1.0; parity tests. 1M rows per side: 12.7 s (pandas) or 11.1 s (SQL), a single run on one desktop.
LLM Support Desk (Python, Ollama, SQLite, Docker)	Free-text payment claims need decisions that follow policy. A model's answer is unreliable and can be injected.	The local LLM proposes; deterministic guardrails decide and can only escalate. JSON schema whitelist, keyword fallback, SQLite decision log, offline eval.	133 tests, 99% line and branch coverage, injection tests. 23 labeled tickets: `llama3.2:1b` 13/23 category agreement, keyword fallback 21/23; 9 of 10 must-escalate tickets escalated.
Document Layout Automator (C#, .NET 8, WPF + CLI)	One extra character in a fixed-width file shifts every later field. Padding without length checks truncates or overflows silently.	Declarative layout profiles; an oversized value rejects its row. Per-row errors with line, field and reason. Fail-closed write shared by GUI and CLI.	38 xunit tests, including golden tests on the committed sample output. Builds with `-warnaserror`, 0 warnings.
Collections Aging SQL (DuckDB SQL, thin Python harness)	Each open invoice must age from its due date and get the right notice. Loose parsing, cash applied out of order and duplicated export ids hide what is owed.	Staging demands an integer-literal amount and casts with `TRY_CAST` into a rejected-rows file, rejects repeated invoice or payment ids after their first occurrence, applies cash FIFO capped per invoice, due-date buckets, escalating dunning stages, duplicate-payment pairs, and three conservation identities (rows, invoice cents, every payment cent in exactly one bucket).	37 tests, golden tests on all seven output files. In CI: ruff, mypy, 85% branch-coverage gate, byte-for-byte output check, exit 2 on findings.

HOW IMPACT WOULD BE MEASURED (NOT MEASURED YET)

- Reconciliation: share reconciled automatically, findings confirmed by finance, time from file arrival to reviewed report.
- Support desk: escalation recall, over-escalation and rule-override rate on labeled real tickets.
- Layout automator: files blocked before upload, ingestion errors downstream, time per file against the manual process.
- Collections aging: notices sent on the right day, duplicate candidates confirmed, time to a call sheet.

SKILLS SHOWN IN THE CODE

- Data and SQL: pandas, DuckDB (CTEs, window functions, `FULL OUTER JOIN`, gaps-and-islands, `TRY_CAST`, running-sum FIFO), SQLite (`json_each`, `NTILE`/`RANK`), integer-cent money, conservation checks.
- Applied LLMs: local Ollama, versioned prompt, JSON output contract, deterministic guardrails, adversarial tests, offline eval.
- Desktop and engineering: C# .NET 8 (WPF) and a CLI, xunit and pytest with coverage gates, ruff, mypy, GitHub Actions, Docker.

BACKGROUND (CONTEXT, NOT IMPACT OF THESE TOOLS)

Billing Specialist at Central Transport (Monterrey, Nov 2023 – Jul 2025): 240+ bills of lading a day and 1,000+ invoices a month, billing accuracy 99%+ (figures from my CV). The projects were built after that role and were not used there. The synthetic data uses Mexican payment rails (OXXO, SPEI, cards) as its setting, with no company affiliation. Built with the help of AI coding tools; each project README documents its design decisions, tests and known limitations.